between different classes—they are all of type object.

For differentiating between classes, TypeScript provides instanceof type guards. An instanceof type guard behaves the same way as typeof and works on any type that is created with a constructor. So we can do something like this:

```
const area(thing: Square | Triangle | Circle): number {
  if (thing instanceof Square) {
    return thing.width * thing.height
  } else if (thing instanceof Triangle) {
    return thing.width * thing.height * 0.5
  } else if (thing instanceof Circle) {
    return PI * 2 * thing.radius
  }
}
```

The important bit here is that inside each if block, we can use attributes specific to each class, like width or radius, because TypeScript uses instanceof to infer the type.

This doesn't exactly solve our reducer issue because our reducer actions aren't created with JavaScript constructors, they are just JavaScript literals. TypeScript provides a different way to type guard generically on the existence of a particular field in the type.

We can use the in keyword to test for the existence of a particular attribute in the object, as shown here:

```
const area(thing: Square | Triangle | Circle): number {
  if ("radius" in thing) {
    return PI * 2 * thing.radius
  } else {
    return thing.width * thing.height
  }
}
```

What's happening here is subtle, and this code actually has a bug when compared to the previous code. The in operator returns true if the object on the right contains the attribute on the left. From a typing perspective, this also acts as